Group 9: Vending Machine Simulator Design Document

Grewal Gaurav
Nwachukwu Obinna
Mugunthan Vaaman
Elashaal Musa
Aghbari Kareem
Powers Casey

1. Introduction & Overview

This document describes the design of an object-oriented vending machine simulator implemented in C++. The simulator models a 10-row by 8-column vending machine with two operating modes (vending and restock), a realistic coin-handling mechanism, and both manual and automatic simulation modes. The simulator uses file-based input for configuration and pseudo-random number generation for reproducible automatic simulations.

2. System Architecture & Class Design

- **What goes here:** This is where you include your **UML Class Diagram** . You must describe the primary C++ classes, their responsibilities, and how they interact.
- **Sample content:** * `VendingMachine`: The central controller. Manages the overall state (Door Open/Closed) and delegates tasks.
    - `SpringCoil`: Represents a single inventory slot. Contains `price`, `itemCount`, `capacity` (max 10), and `itemID`. (An 8x10 2D array or vector of these will live in the `VendingMachine`).
    - `CoinMechanism`: Manages money. Contains sub-components or logic for the `ChangeBox` (maps coin denominations to quantities) and `CashBox` (total deposited earnings).
    - `Simulator`: Handles the explicit representation of time, reading the file input for simulation parameters, and managing the event queue.

3. System States & Flow Diagrams

Customer Vending Flow

Shows how a customer interacts with the machine. This includes inserting coins, selecting an item, and either receiving their item with change, getting an error message, or cancelling and getting their coins back.

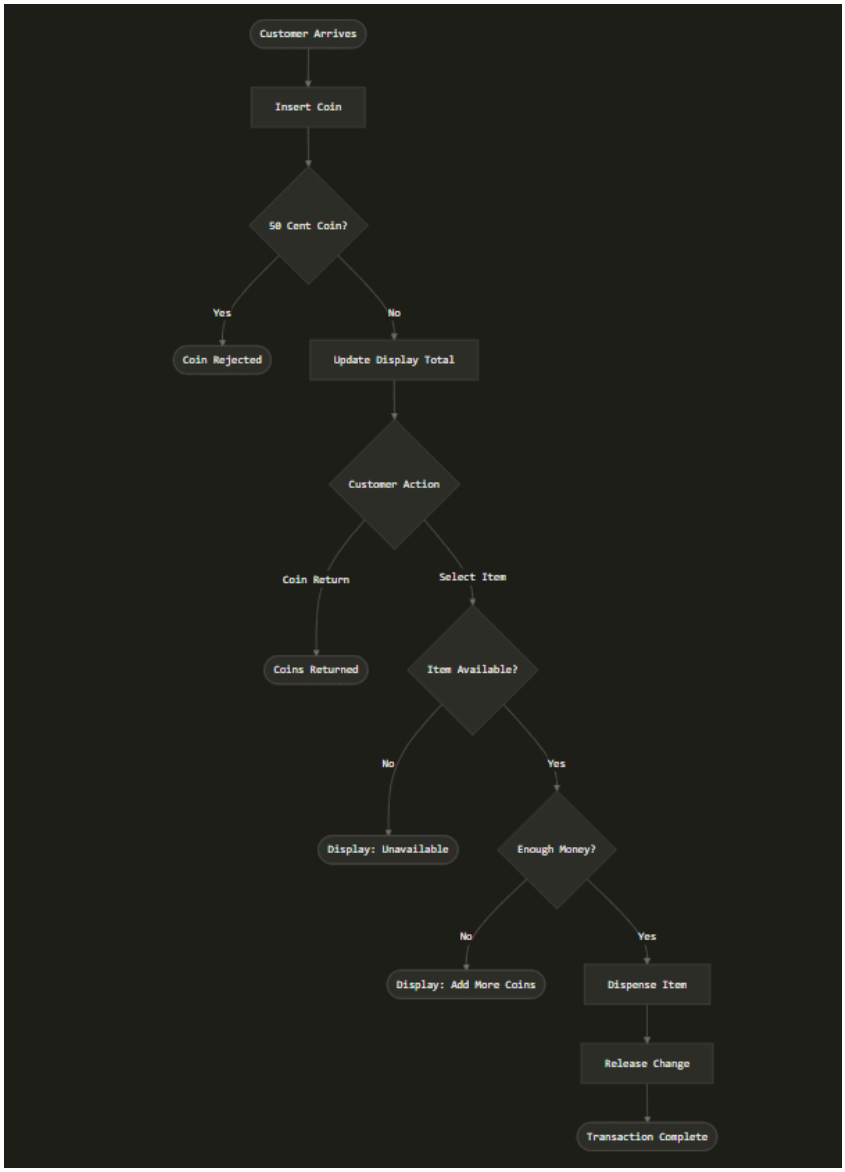

Fig 1. Customer Vending Flow

Service Worker Restock Flow

Shows how a service worker restocks the machine when the door is open. This includes setting prices, refilling coils, managing the cash and change boxes, then closing the door to return the machine to vending mode.

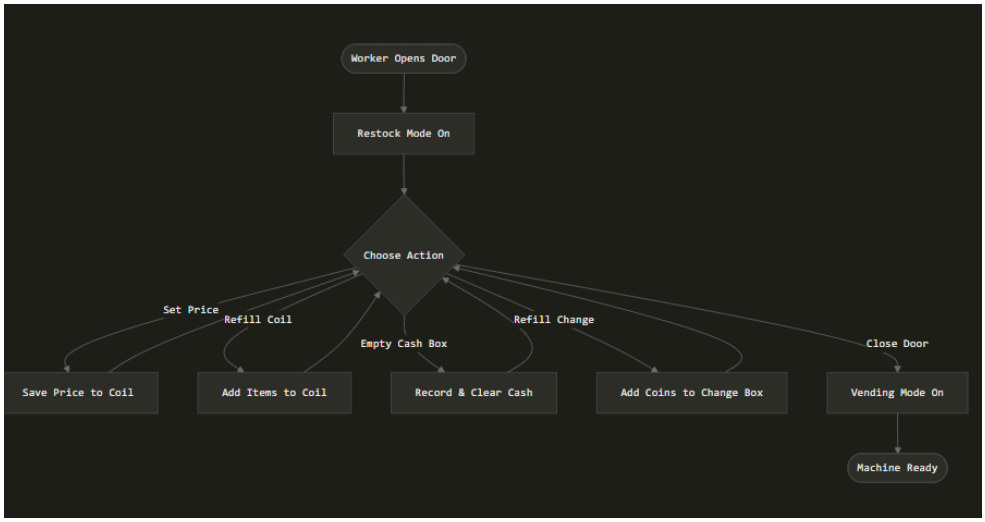

Fig 2. Restock Flow

Automatic Simulation Flow

Shows how the simulation automatically generates random customer and worker arrivals, runs them through their respective flows, logs the results to statistics, and repeats until the simulation ends and a final report is printed.

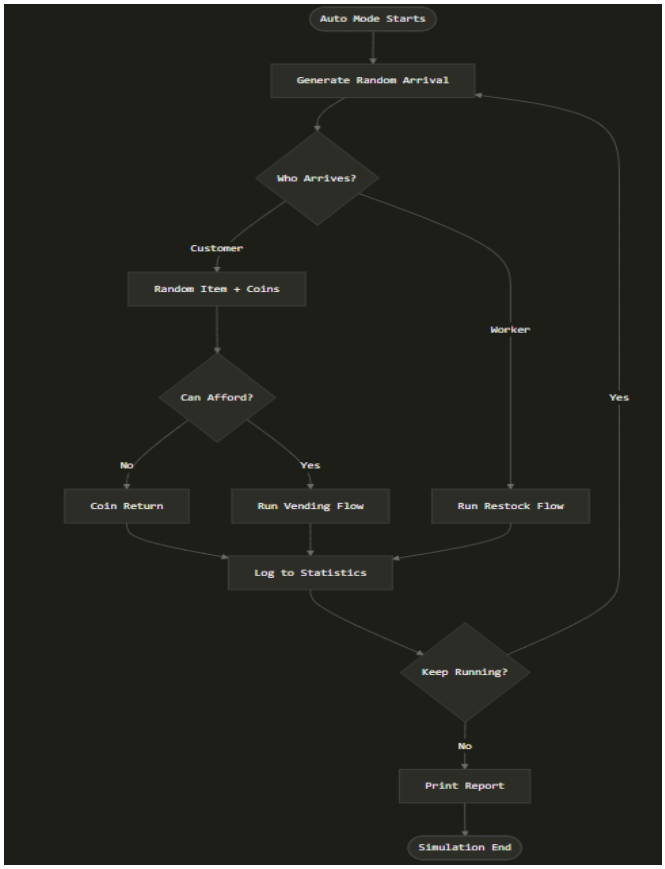

Fig 3. Simulation Flow

4. Algorithm Descriptions

Coin Insertion Algorithm
When a customer inserts a coin, the machine first checks if the door is open. If so, the coin is immediately rejected, else the coin’s type is identified. If it is a 50 cent coin, it is rejected and returned to the user using the coin return slot. Else, the coin's value is added to the customer’s current running total and the display is updated in real time.

Item Selection Algorithm
When a customer presses a row button (A-J) followed by a column button (1-8), the machine records the selection and displays it. To enforce correct inputs, this is put in a while loop until the expected and allowed inputs are given. The machine then checks the corresponding coil. If it is empty, a “item unavailable” message is shown and no further action is taken. If the item is available, it checks whether the customer’s entered total is greater than or equal to the item’s price. If not, an "insufficient funds” message is displayed and the system goes back to allowing more coins. Finally, if both tests pass,the machine dispenses the item and calculates the change.

Dispensing and Change Algorithm

Once a valid purchase is confirmed, the spring coil releases the front item and the item count is decremented by one. The machine then calculates the change owing as the difference between the amount entered and the item's price. Starting from the largest coin denomination down to the smallest (toonie, loonie, quarter, dime, nickel), the machine checks how many of each coin are available in the change box and uses as many as needed without exceeding the change owing. Each coin directed out of the change box is dropped into the coin return slot. The customer's running total is then reset to zero. If the change box cannot make an exact change, this case is handled by the low-change check described below.

Coin Return Algorithm

When the customer presses the coin return button, the machine takes all coins currently held from the current transaction (those not yet dropped into the cash box) and routes them back through the coin return slot. The customer's running total is reset to zero and the display returns to its idle state.

Low Change Detection Algorithm

After every transaction, the machine evaluates the total remaining value in the change box. If the value falls below a defined threshold, the display shows an "exact change only" message. This check is also run at startup. When exact change only mode is active, the machine will only

complete a transaction if the amount entered is exactly equal to the item's price, ensuring no change is needed.

Automatic Simulation Algorithm

When the simulation is set to automatic mode, the controller uses a random number generator to determine when the next customer arrives, based on a configurable average arrival rate. When a customer arrives, they randomly select a row and column, then insert randomly generated coin combinations up to a maximum spending threshold. If the item is too expensive relative to what they are willing to spend, they press coin return and leave. Otherwise they complete the purchase. Separately, a restocking worker arrives at random intervals, opens the door, refills all coils that are below a threshold, replenishes the change box, empties the cash box, and closes the door. After each simulated event, statistics are updated.

Statistics Tracking Algorithm

Throughout the simulation, the statistics module records the following after each relevant event: total revenue collected, number of items dispensed per coil, total change dispensed, number of times "exact change only" was triggered, number of times a selection was unavailable, and number of times a customer had insufficient funds. On request, the statistics module formats and prints a summary report of all recorded values, which can be used to analyze machine performance and determine appropriate starting change box levels.

5. Simulation Mechanics & Statistics Collection

- **What goes here:** Address the requirement to "explicitly represent time and state changes" and "use file input."
- **Sample content:**
    - **Time & Events:** Explain your simulation loop (e.g., a discrete-event simulation using a priority queue of events sorted by timestamp).
    - **File Input:** Describe the format of the input file (e.g., `seed_value, arrival_rate, initial_stock`).
    - **Statistics:** List the exact metrics you are tracking. Examples: *Total items sold, total revenue, number of times the machine ran out of a specific item, number of times exact change could not be made, average time between restocks.*

6. Key Assumptions & Design Decisions

- **What goes here:** Justify *why* you built it the way you did, and outline any boundaries you set for the simulation to make it manageable.
- **Sample content:**
    - *"**Assumption:** Customers are willing to wait indefinitely for the machine to process their transaction, meaning no timeouts occur mid-purchase."*

- *"**Decision:** We use the `<random>` library with a Mersenne Twister (`std::mt19937`) initialized by the seed from the input file to ensure 100% reproducibility."*
- *"**Assumption:** The Restocker always brings enough inventory and coins to completely fill the machine and change box."*

7. Workload Distribution

- **What goes here:** A clear breakdown of who did what. Since individual contribution is worth 20% and GitHub history is checked, this table must align with your commit history.
- **Sample content:** | Team Member | Primary Responsibilities | | :--- | :--- | | **Student A** | Designed `CoinMechanism` & `ChangeBox` logic; Implemented the greedy change algorithm; Handled file I/O. | | **Student B** | Built the main `VendingMachine` and `SpringCoil` classes; Implemented Restock Mode functionality. | | **Student C** | Built the `Simulator` event loop; Handled PRNG integration; Wrote statistics tracking and output. |